HomeWork

px?

sp?

dp?

In Android development:

1. **px** represents physical pixels.
2. **dp** (density-independent pixels) scales with screen density.
3. **sp** (scale-independent pixels) scales with user's font size preference, primarily used for text sizes.

Set font

`Text` has a `fontFamily` parameter to allow setting the font used in the composable.

By default, **serif**, **sans-serif**, **monospace** and **cursive font** families are included:

```
@Composable
fun DifferentFonts() {
    Column {
        Text("Hello World", fontFamily = FontFamily.Serif)
        Text("Hello World", fontFamily = FontFamily.SansSerif)
    }
}
```

Output:

Hello World

Hello World

Button

Buttons are fundamental components that allow the user to trigger a defined action. There are five types of buttons. The following table describes the appearance of each of the five button types, as well as where you should use them.

The following image demonstrates the five types of buttons in Material Design.

Figure 1. *The five button components.*

Type	Appearance	Purpose
1. Filled	Solid background with contrasting text.	High-emphasis buttons. These are for primary actions in an application, such as "submit" and "save." The shadow effect emphasizes the button's importance.
2. **Filled tonal**	Background color varies to match the surface.	Also for primary or significant actions. Filled buttons provide more visual weight and suit functions such as "add to cart" and "Sign in."

3. **Elevated**	Stands out by having a shadow.	Fits a similar role to tonal buttons. Increase elevation to cause the button to appear even more prominently.
4. **Outlined**	Features a border with no fill.	Medium-emphasis buttons, containing actions that are important but not primary. They pair well with other buttons to indicate alternative, secondary actions like "Cancel" or "Back."
5. **Text**	Displays text with no background or border.	Low-emphasis buttons, ideal for less critical actions such as navigational links, or secondary functions like "Learn More" or "View details."

API surface

- **`onClick`**: The function called when the user presses the button.
- **`enabled`**: When false, this parameter causes the button to appear unavailable and inactive.
- **`colors`**: An instance of `ButtonColors` that determines the colors used in the button.
- **`contentPadding`**: The padding within the button.

Filled button

The filled button component uses the basic `Button` composable. It is filled with a

```
@Composable
fun FilledButtonExample(onClick: () -> Unit) {
    Button(onClick = { onClick() }) {
        Text("Filled")
    }
}
```

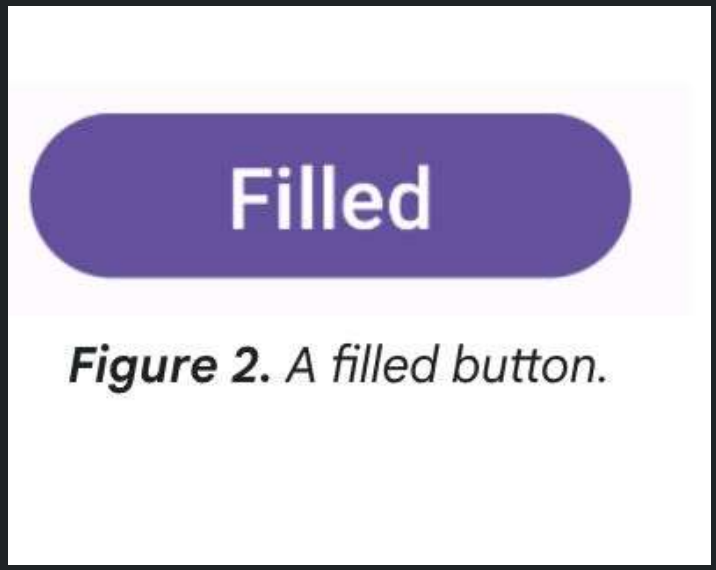

Figure 2. *A filled button.*

Filled tonal button

The filled tonal button component uses the `FilledTonalButton` composable. It is filled with a tonal color by default.

The following snippet demonstrates how to implement the component:

```
@Composable
fun FilledTonalButtonExample(onClick: () -> Unit) {
    FilledTonalButton(onClick = { onClick() }) {
        Text("Tonal")
    }
}
```

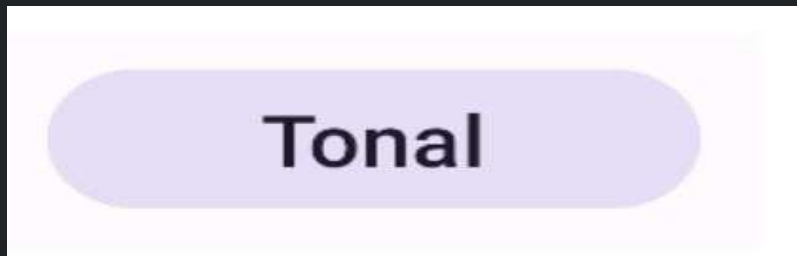

Figure 3. *A tonal button.*

Outlined button

The outlined button component uses the `OutlinedButton` composable. It appears with an outline by default.

The following snippet demonstrates how to implement the component:

```
@Composable
fun OutlinedButtonExample(onClick: () -> Unit) {
    OutlinedButton(onClick = { onClick() }) {
        Text("Outlined")
    }
}
```

Figure 4. *An outlined button.*

Elevated button

The elevated button component uses the `ElevatedButton` composable. It has a shadow that represents the elevation effect by default. Note that it is essentially an outlined button with a shadow.

The following snippet demonstrates how to implement the component:

```
@Composable
fun ElevatedButtonExample(onClick: () -> Unit) {
    ElevatedButton(onClick = { onClick() }) {
        Text("Elevated")
    }
}
```

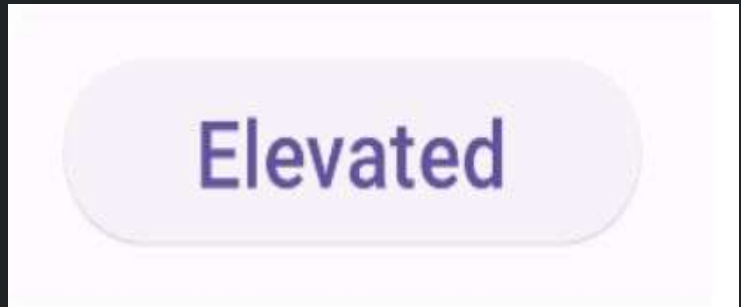

Figure 5. *An elevated button.*

Text button

The text button component uses the `TextButton` composable. Until pressed, it appears as only text. It does not have a solid fill or outline by default.

The following snippet demonstrates how to implement the component:

```
@Composable
fun TextButtonExample(onClick: () -> Unit) {
    TextButton(
        onClick = { onClick() }
    ) {
        Text("Text Button")
    }
}
```

Figure 6. *A text button.*

Checkbox

Checkboxes let users select one or more items from a list. You might use a checkbox to let the user do the following:

- Turn an item on or off.
- Select from multiple options in a list.
- Indicate agreement or acceptance.

Anatomy

A checkbox consists of the following elements:

- **Box**: This is the container for the checkbox.
- **Check**: This is the visual indicator that shows whether the checkbox is selected or not.
- **Label**: This is the text that describes the checkbox.

States

A checkbox can be in one of three states:

- **Unselected**: The checkbox is not selected. The box is empty.
- **Indeterminate**: The checkbox is in an indeterminate state. The box contains a dash.
- **Selected**: The checkbox is selected. The box contains a checkmark.

The following image demonstrates the three states of a checkbox.

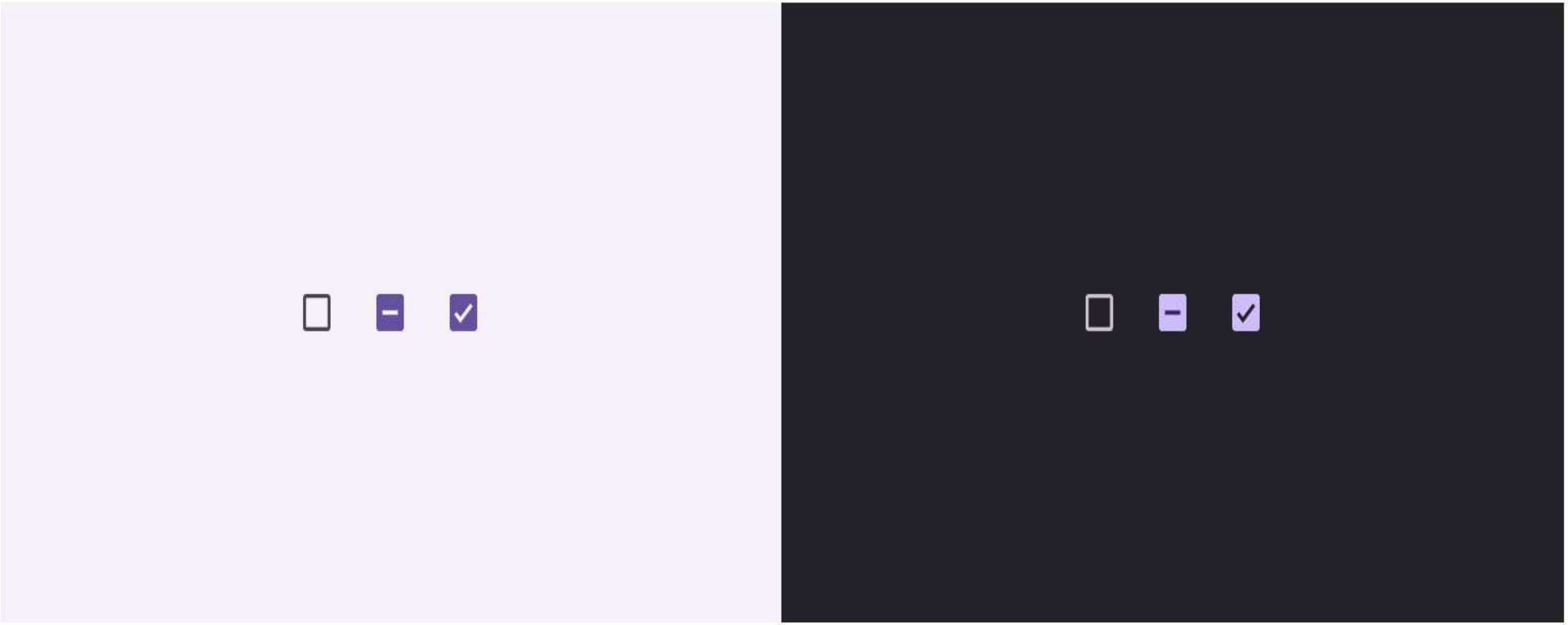

Figure 1. *The three states of a checkbox. Unselected, indeterminate, and selected.*

We can use the `Checkbox` composable to create a checkbox in your app. There are just a few key parameters to keep in mind:

- **`checked`**: The boolean that captures whether the checkbox is checked or unchecked.
- **`onCheckedChange()`**: The function that the app calls when the user taps the checkbox.

```
@Composable
fun CheckboxMinimalExample() {
    var checked by remember { mutableStateOf(true) }

    Row(
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Text(
            "Minimal checkbox"
        )
        Checkbox(
            checked = checked,
            onCheckedChange = { checked = it }
        )
    }

    Text(
        if (checked) "Checkbox is checked" else "Checkbox is unchecked"
    )
}
```

Output

This example produces the following component when unchecked:

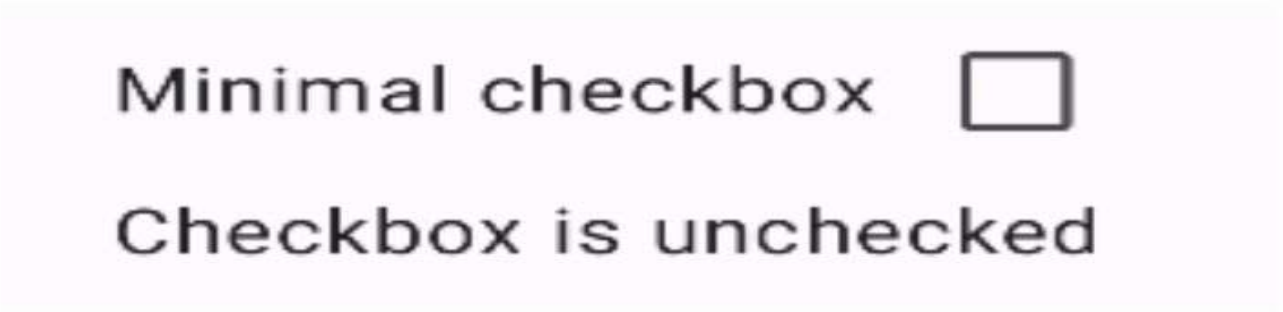

Figure 2. *Unchecked checkbox*

And this is how the same checkbox appears when checked:

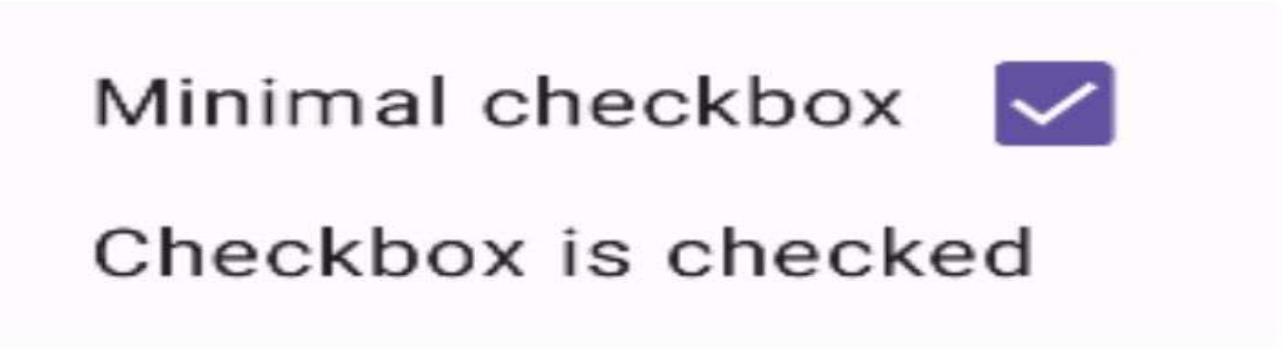

Figure 3. *Checked checkbox*